

Staying Agile in Government Software Projects

Barg Upender
SRA International, Inc.
barg_upender@sra.com

Abstract

Can government software projects be agile? What do Scrum and XP practices have to offer in this regulated and highly political environment? In this experience report, I will discuss some of the unique challenges in our environment and how we have had to adapt these practices to produce commercial-grade software. I will provide a "report card" on our progress in applying Scrum and XP practices to a clinical data management project over a two-year period. In addition, I will describe the practices that were accepted "religiously", adapted to get the job done, and abandoned completely. In particular, I will discuss how we got around bootstrapping, Rational tools, documentation needs, and managing a product backlog for a diverse, decentralized user community. Putting these practices to work was hard, but they resulted in better team communication, a more usable product, and improved partnership between the users and the development team. In addition, staying agile is just as hard as becoming agile.

1. Introduction

Scrum has its origins in the competitive world of commercial software product development [1]. This paper discusses our experience with applying the Scrum practices on a software project in a Federal contracting environment. The project's goal was to develop a data and workflow management system for scientists conducting clinical research at the National Institutes of Health (NIH). Clinical research involves studies on human subjects in order to find better ways to detect, diagnose, treat, and prevent a variety of diseases. Since these studies involve humans, the highest ethical and safety standards must be followed.

The clinical research lifecycle involves proposing a hypothesis, developing a research plan, review of the plan by a safety review board, conducting the study, analyzing results, and publishing the findings. The proposed software would be used by the researchers to reduce paper, improve the data collection process, and simplify data analysis. The three-year project is divided into three overlapping phases:

1. Automate the paper-intensive safety review process
2. Electronically capture clinical research data
3. Integrate patient data with other databases and provide tools to analyze data across studies.

Each phase is roughly one year. Currently we are 2 years into the project.

The Team

The project is a collaborative effort between two NIH institutes. The sponsoring institute contributes a business analyst and a software developer. The second institute contributes four developers and a project manager. In total, the core development team consists of four Java developers and an Oracle database administrator. The development team is from SRA International, a leading government contractor. The project manager is a federal employee who is accountable to the sponsor. The sponsor reports to a steering committee consisting of research scientists.

The user base is made up of research groups consisting of a Principal Investigator, medical doctors, nurses, post-doctoral fellows, and administrative assistants. These groups are organized to operate independently in order to encourage creative and competitive research (similar to a University environment). Each group has their own unique study process, data collection approach, and rudimentary tools (e.g. Excel, Access) to capture data. The key objective of the project is to replace these point systems with a central, enterprise-class data management system.

There had been several failed attempts at solving this problem in the past due to the inherent challenges of unifying the requirements and the organization. The traditional waterfall-based contracting model was not able to address the technical and organizational challenges.

Our team is composed of senior developers with broad experience in software product development. The team had a strong commitment to project success and was eager to gain the customer's confidence.

Project Vision

During the project kickoff meeting, the sponsor outlined the broad capabilities of the system. The system should allow the researchers to design their own data collection forms, provide secure access to patient data, and provide an audit trail for regulatory compliance. The project manager presented the development approach and outlined the development tools to be used.

Much of the software development work at federal agencies is done by contractors. Contractors come and go and the knowledge gets lost. To preserve the requirements, business rules, and design decisions in such a dynamic environment, the project manager wanted to use the Rational Unified Process (RUP) and the Rational toolset, including RequisitePro for managing requirements, XDE for modeling the system in UML, ClearQuest for configuration management, and IBM WebSphere Studio for Integrated Development Environment (IDE). Several of these tools were already purchased. In a previous company, I had successfully used Rational Rose to migrate to object oriented technology. Iterative, model-based software development made engineering sense. The team was enthusiastic about learning the tools and putting these ideas into practice.

Phase 1 Approach

The initial milestone was to produce the Safety Review Management application in six months. Since the team was new to the toolset and the clinical research domain, we decided to incrementally inject the tools into the development process. First, we focused on setting up a strong development environment based on the best practices. Since we had better experience with CVS than ClearQuest in the past, we were able to convince the management to adopt CVS. We installed CVS to manage the source code, configured a WebSphere development environment, and adopted our corporate coding standards. For the time being, we decided to follow a simplified Rational Unified Process with a rhythm consisting of monthly iterations. The UML modeling effort was deferred until Phase 2.

As the problem domain was new to the team, the first three months were spent understanding the requirements and learning the new tools. The team generated use cases, requirements, architecture, and component design documents. We had scheduled team meetings on Tuesdays to discuss technical issues. Often these meetings ran long, and they quickly became unpopular. We also had scheduled Friday meetings with the sponsor to provide status, show a demonstration, and obtain feedback.

After an initial meeting with the users, the team did not have direct access to the end users. The information was conveyed through the business analyst. The first few “iterations” consisted of developing a static HTML-based prototype of the requested functionality. The screens used fake data. The sponsor and the users interacted with the prototype and provided early feedback.

Within the six months, we delivered a system with the key features. However, it took eleven months total elapsed time and six more releases before the users started using the system. Several factors contributed to this delay:

- As users interacted with each release, requirements changed slightly and new requirements emerged. The users did not want to use the system until these features were completed.
- The users were extremely busy with the existing paper and Excel-based process.

The development team continued to add the requested features to the system.

2. Road to Agility

The second phase of the project, which we are completing in this fall, has proven to be significantly more challenging. In this phase, we have to manage users with diverse needs, build a web-based form design tool, and provide mechanisms to share common data elements in a decentralized environment. In anticipation of these challenges, I started looking around for methods and tools that would help us with managing requirements, planning releases, designing the application, and facilitating communication in the team.

In the fall of 2003, I attended the Software Development Conference in Boston. Due to the .com crash and the ensuing recession, I hadn’t been to a trade conference in several years. At the conference, half the talks focused on RUP and the other half focused on emerging agile methods. In particular, Joshua Kerievsky’s Industrial XP [2] and Ken Schwaber’s Scrum [1] presentation had a profound influence on me. These talks resonated with me, as they pertained to some of the difficulties I was facing as well as potential ways to tackle these challenges. In particular, the value-driven, evolutionary design approach of IXP looked promising. This technique would allow the team to focus on high-value items first and defer uncertain requirements until they became clearer. However, IXP was a more radical concept for the management and team. Scrum was simpler to understand and more “management friendly”.

After returning from the trip, I presented the ideas to the team. Both the management and the team were skeptical about these practices. The Rational Unified Process still made good engineering sense and seemed to them more predictable and controllable. Meanwhile, the agile manifesto [3] sounded like some left-wing, anti-government document. Surprisingly, there was even more resistance from the developers. I thought the developers would be empowered by the self-organizing team approach. But, most of the developers were talented, highly-seasoned engineers and there were a lot of philosophical design and process differences. Shifting them to the agile mindset would take time.

Phase 2: First Agile Cycle

Since the problem domain was getting more complex, the design meetings were getting intense and the requirements remained conflicting and unclear. We decided to do a quick, one-week development cycle using an evolutionary design approach. The goal of this iteration was to develop an “embryonic” clinical application that provided coarse end-to-end functionality. The application would allow the user to design a simple form with questions, fill out the form, and export the form data. To keep it manageable, we deferred implementation of patient management, support for multiple user groups, and authentication screens to future releases.

Generally, the cycle was a mini-success. The team was pleased to be delivering functionality rather than having huge philosophical design discussions and producing documentation. However, the agile ideas and practices didn’t stick. The management was concerned that the “big” problems in the system weren’t solved and we shouldn’t be developing code until we had a better handle on these solutions. The management asked questions like “What is the big picture?,” “What new technologies do we need to explore and adapt?,” and “When will the application be done?”

As a result, management decided that the team’s time would be better spent investigating technologies for addressing the challenging problems. At this point, we went into “research and demo mode” and abandoned all development. Over the next couple of months, each developer explored and presented different technologies, including: Microsoft InfoPath for designing forms, XForms for generating web forms, enterprise portals to provide a consistent interface, clinical data standards to leverage existing designs, and medical vocabularies to integrate data.

Scrum Training

In February 2004, I was able to encourage the project manager to take Ken Schwaber’s Scrum training in nearby facilities in Philadelphia. We got up at four in the morning and drove four hours to Ken’s training at Primavera Systems Inc. Due to traffic conditions in the various cities we passed, we arrived ten minutes late to Ken’s training. We had to pay a dollar each to the “pizza fund”; thus started our Scrum training. At the end of the training, I had the management buy-in. However, to minimize controversies, the project manager decided not to “flaunt” Scrum terminology and incrementally adopt these practices over time. Over time, the short release cycles, working software and user acceptance allowed both management and developers to embrace the agile model. Now, the team corrects me if I stray from the agile way. It took us six months to adopt several agile practices.

3. Adopting and Adapting Scrum Practices

Scrum practices provide practical methods to prioritize requirements, plan iteration, and complete a development cycle. The following section describes how we implemented and adapted these practices.

Product Owner

In Scrum, the product owner is typically a single person who is responsible for setting the priorities and managing the requirements. Often, this person is from the sales and marketing department and would have a good handle on the needs of the users.

Our project had ambitious goals with a high degree of uncertainty in the requirements. The application needed to please diverse user groups with limited patients. Some of the groups were large, with over fifty users and many years of clinical research data. Some groups were small, with more therapeutic-based research (i.e. clinical trials) where there is a more well-defined process mandated by the Food and Drug Administration (FDA). Some groups had a variety of data types (images, wave forms, and lab data) with sophisticated analytical needs. Yet other groups had elaborate screening and recruitment procedures. Our first challenge was to prioritize these requirements and incrementally roll out the system to the users.

Having a single product owner proved to be difficult in our environment. It was difficult for a single person to understand the intricacies of the end state of the application. A representative from both institutes was needed. Consequently, we established a team of three people to prioritize features and make decisions. It

consisted of the project manager, the senior developer from the other team, and me (the Scrum master).

User Backlog

First, we prioritized the user groups based on their immediate needs and internal politics. The evolving system would be rolled out to one user group at a time. Generally each user group would get one or two releases. In essence, we developed a “user backlog”. Figure 1 shows a partial backlog for the first three releases. The horizontal axis is the timeline. The vertical axis shows the releases. Each release, represented by a box, is developed for an investigator and is focused on a functional theme. This backlog provided a roadmap for prioritizing the product backlog.

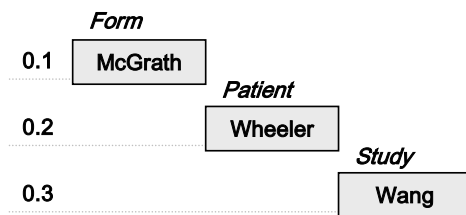


Figure 1: An example user backlog

The users were tired of new teams coming to them, gathering requirements, and coming back in year with completely unusable system (or even never coming back). The users did not trust developers. To regain their trust, we met with the users, explained our process, demonstrated our preliminary system, and captured their feedback on a Wiki to share with the rest of the team. We copied the urgent, high-values stories into the product backlog, planned and completed a sprint, and went back to the users to show them the new features.

The investigators were pleased with the working software and the fact that we listened to their needs. They were not accustomed to such a quick turn around. They got excited about the system and provided more guidance. In fact, our first user group cancelled an existing multi-year contract from another vendor and adopted our system. Our system provided more features in three months than the vendor was able to accomplish in eighteen months.

Product Backlog

In phase 1, we had captured the product requirements in an Excel spreadsheet. Neither managers nor developers read this document. These requirements were helpful in planning our monthly releases in Microsoft Project. In Phase 2, we briefly tried Rational RequisitePro and abandoned it since it introduced additional barriers for the team to access requirements. We also tried index cards used in XP practices [4]. However, the management had

a strong desire to capture the knowledge in something more permanent. In the end, we settled on Bugzilla, our defect tracking system.

Bugzilla became our single repository to manage both product backlog and bugs encountered in the field. Bugzilla also helped us with our workflow and communication. For example, it helped us manage the bug owner, bug state, as well as email communications. We also annotated each “bug” with requirement clarifications, design notes, data model changes, and other relevant information. After design discussions, we took digital pictures of the whiteboard and put them into Bugzilla. On occasions, we went back to the attachments for clarification. Bugzilla not only helped us with managing the product backlog, but also helped the address the project manager’s documentation concerns.

Bugzilla’s simple search and reporting capabilities helped us create a “burn-down” chart, which identifies the amount of remaining work. The Bugzilla tool is seamlessly integrated with the project Wiki. The Wiki contains informal notes such as sprint dates, design notes, release notes, and the burn-down chart. The combination of a Wiki with Bugzilla works well; we abandoned the costly tools and adopted the free, flexible ones. We spend very little time on these tools; our focus is on delivering working software.

Sprint Planning

In Scrum, each sprint is time boxed to thirty days. During an initial meeting the product owner defines the highest priority features to the team. Then, the team has a second four-hour meeting to breakdown the features into tasks, produce an estimate to complete the tasks, and make commitments to the tasks that they think they can deliver.

Generally, our planning process is not as efficient as the official Scrum practice. For the simple features, coming up with estimates is fast and easy. However, we wrestle with some of the more challenging features that impact the core data model. We explore different design ideas in an attempt to maintain a balance between getting something done for this release versus creating a more lasting solution. Most senior designers are instinctively driven to big designs that they feel will last forever. But, I am strong believer in YNGNI, “you’re NOT gonna need it” [5]. Ultimately, it’s a balance between over-design and sloppy design. I think achieving this balance is the single most important factor in determining the level of agility a team can have.

In the end, we couldn’t complete the planning process until we had a rough design approach. Consequently, our planning process takes several days to complete.

However, we continue developing code in parallel for the easier features while we finalize the plan.

We tried different estimations and tracking approaches, including NUTS (nebulous units of time) as proposed by IXP. Generally, developers didn't like estimating. In the end, we just based the estimates on actual time. Kent Beck also suggests this practice in the new version of the Extreme Programming book [6].

For a couple of sprints, we had separate spreadsheets for the product and sprint backlogs. However, when we fully migrated to Bugzilla as the product backlog, our sprint backlog just became a subset of the product backlog. We just added details to existing bugs or created new bugs as needed. We didn't have to maintain a separate list for the sprint.

Because there is a pressure to produce big things (high-value) for each release, sometimes the little, annoying tasks always remained low on the priority list. These tasks take several iterations before they are planned into a sprint. Often these tasks are related to user interaction design and user satisfaction.

Sprint Management

During the sprint, the Scrum master shields the team from external interferences. The team achieves high bandwidth communication and makes all decisions to support the sprint goals.

The sprint executed smoothly once the team agreed on the high-level designs and the tasks in a sprint. In the following section we will discuss three key aspects of sprint management: daily Scrum, just in time design, and sprint burn-down.

Daily Scrum

Daily standup meetings are one of the hallmarks of Scrum. I expected a lot of resistance from the team as it may seem like over-management. Surprisingly, this was our first successful practice.

Our project involved collaboration with a developer in another institute, located in a different building on the campus. There were communication bottlenecks between the institutes. We introduced daily Scrum at 10am to make commitments and to remove obstacles. The developer from the other institute dialed in most of the time. When a tighter coordination was required, the developer walked over to the meeting.

As we had no open space for daily meetings, we met in the project manager's office for the stand-up meeting. Reserving conference rooms was a lengthy process.

Eventually, we were able to get a dedicated conference room for our daily Scrums.

We kept the meetings semi-formal and fun to facilitate team building, provide a daily synchronization point, and help developers communicate more. There were no late fees and we had sit down meetings. If an issue required detailed discussions, we had follow-on meetings with a smaller team. We also used Instant Messenger (IM) often and VNC as needed.

On several occasions we postponed the meeting to wait for an employee who needed to come in late. However, that didn't go very well because the team's rhythm was disrupted. In the end, we went with simple, consistent rules – start on time with whoever is available. The daily Scrum is crucial, as it keeps the team informed and focused.

Just in time design

As mentioned earlier, “thinking light” about the design was one of the hardest things that the team had to confront. There were strong and passionate opinions about the optimal way of designing the application. Often the disagreements are due to the variety of perspectives that developers brought to the team (e.g. data model, security, and usability perspectives).

In addition, the clinical research domain is complex. We are developing a centralized system for an organization that is inherently decentralized. For example, the investigators are not used to trying to use common vocabularies, common data collection forms, and common study processes. So the requirements are always in flux and often in conflict.

We have tried an evolutionary design approach, and the strongest design ideas have survived over the iterations. We had to abandon some bleeding-edge technologies (e.g. Microsoft Infopath, XForms, Oracle XMLDB) after repeated attempts to smoothly integrate with our system. Making these decisions has been hard, but things have progressed smoothly afterwards.

Burn-down chart

For a few sprints, we maintained a burn-down chart in Excel. The spreadsheet and the chart tracked hours remaining for each task. The team just cared about making steady progress, so tracking the chart to hours remaining wasn't needed. In the end, we went with a simpler approach of tracking how many “bugs” (i.e. tasks) were completed in the last 24 hours, and how many “bugs” remained. I posted these numbers on the Wiki. This provided the visible progress that we needed to assess how we were doing in achieving our sprint goals.

Sprint Review

At the end of a sprint, Scrum recommends an informal meeting that demonstrates the functionality to the product owner and other key stakeholders. The team reflects on what worked and what didn't work.

In our project, we had an internal demo with the team and the project manager. This allowed the whole team to look at the finished product. During this process, we often discovered some rough spots in the navigation, which are often easy fixes. If a fix is more involved, we put it in the product backlog for next release.

The product owners also scheduled separate demonstrations for the sponsor and the different user groups. There was a strong desire by most of the development team to observe how the users interacted with the application, but the whole team could not be involved in these demonstrations due to space limitations, and also to avoid overwhelming the users. Unfortunately, identifying which team member can attend these meetings became contentious. We resolved this by having each developer "adopt" a different clinical researcher (i.e. user group) on a rotating basis. The developer goes to the meetings with the product owners. The developer is also responsible for setting up the demo data (e.g. creating forms and reports), and putting the notes on the Wiki for the others to view.

After the release is installed on the production environment, we have a team lunch to celebrate the release. Then, we start a new sprint.

4. Scrum Report Card

It took us six months to get started with agile software practices. Figure 2 shows a self-assessment of how well we follow the Scrum principles. For example, the daily Scrum is one of the more successful practices that we follow with minimal alterations. We also have been efficient at managing the requirements in the product backlog using Bugzilla.

Roles	B	13
Backlog		16
Planning		12
Sprint		15
Daily scrum		16
Review		12
Score		84

Figure 2: Scrum Report Card

Due to the small size of the team, each team member is responsible for multiple roles. For example, the Scrum master and the product owners are also developers. The developers also support the production environment and provide customer support. If we had dedicated resources for these roles, we might follow the process more methodically.

Our planning process is the least efficient as it takes several days. Generally, we can't finish planning until we have agreement on the high level approach. This design process takes time to get everyone on the same page. However, once the sprint is planned, we are good about executing the sprint efficiently.

Other Agile Practices

We have also experimented with various other agile practices. For example, we tried pair programming on several occasions, with positive outcomes. However, due to our small team size, we pair only on the challenging coding problems.

Each developer frequently synchronizes with the CVS repository. We have set up *CruiseControl* to build the application on check-ins and run minimal test cases. This application sends an email to the team if the build or test cases fail. Since we have limited test cases, we are not benefiting from continuous integration as much as we can.

We tried Test Driven Design (TDD) during several sprints. We used jUnit, httpUnit, and Fitnesse. Our application is HTML-intensive. At the web-layer, we developed some unit tests using httpUnit and acceptance tests using Fitnesse. However, these tests were brittle as the user interface changed frequently. At the business object layer, we developed jUnit tests. Much of our business logic consisted of straightforward add/edit/delete transactions. We have limited complex business logic and/or algorithms where extensive test case would have been more justifiable. We ran into two issues: 1) getting the database to a known state; and 2) writing a set of test cases that would find real problems, so that maintaining test cases didn't become a burden. Perhaps, we can overcome these difficulties by using dbUnit and some formal training.

Generally, developing these test cases hasn't seemed to be cost-effective. However, I think with proper training and discipline, TDD can be a valuable practice that can lead to a more focused implementation.

Shared workspace hasn't been an option. Each team member has a cubicle office. We have overcome this

limitation by having lot of large white boards on the outside of the walls where the team can meet to discuss ideas. We also use Instant Messaging effectively.

5. Summary

Overall, the Scrum practices have helped us become more efficient at delivering software. The team does not miss the scheduled weekly meetings, architecture documents, requirements spreadsheets, and variable release cycles.

We have had to adapt some practices to our particular environment. Putting these practices to work was difficult as we were dealing with challenging requirements and strong design philosophies. But agile practices have unquestionably helped the team, the project managers, and the end users. The team has better communication. We spend more time writing software, and are pleased with user acceptance. The project managers are pleased with the team commitment. Daily visibility into progress gives the team more freedom to make decisions. The users are pleased with the quick turn-around and the fact that they have more control over the direction of the system.

Rigidity in the organization and team determines the level of agility that can be achieved. Scrum principles are simple. Internalizing these principles and rigorously following them is hard. Staying agile is just as hard as becoming agile.

6. Acknowledgements

This project has been successful because of the talented and dedicated engineering team; Sarada Chintala, Adam Frazin, Raj Lingam, Wen Nie, and Gladys Wang. I would like to thank the customer and the management for their confidence and support. Additional thanks to David Kane, Marc Kellogg, and Rob Sfeir for their feedback in reviewing the paper. Special thanks to Rebecca Wirfs-Brock for valuable input and patiently shepherding the paper.

7. References

- [1] K. Schwaber, M. Beedle, *Agile Software Development with Scrum*, Prentice-Hall, 2002.
- [2] Industrial XP (Web Site: <http://industrialxp.org/>)
- [3] Agile Manifesto, (Web Site: [http:// agilemanifesto.org/](http://agilemanifesto.org/))
- [4] K. Beck, *Extreme Programming Explained: Embrace Change*, Addison-Wesley, 1999.
- [5] YAGNI, (Web site <http://xp.c2.com/YouArentGonnaNeedIt.html>)

- [6] K. Beck, *Extreme Programming Explained: Embrace Change* (2nd edition), Addison-Wesley, 2004.